ChatGPT

Executive Summary

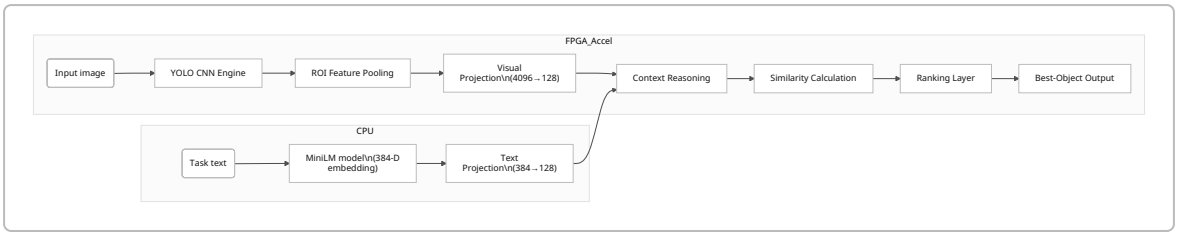
We propose an **edge-friendly pipeline** where the RISC-V *VEGA* CPU encodes the task text (via a MiniLM model) while an FPGA-based accelerator processes the image. The accelerator runs a YOLO object detector to get object features, then performs lightweight task-aware reasoning (projecting features, modeling context, computing text–object similarity, and ranking). This parallel design minimizes end-to-end latency: the CPU and CNN work in tandem, and once both task and image features are ready, the accelerator quickly scores each object against the task. Our analysis shows that **YOLO dominates the compute** (tens of GFLOPs per frame), so its weights reside off-chip in DDR3 memory, while the small reasoning network (few thousand weights) stays on-chip in BRAM. DDR bandwidth (14+ GB/s on Genesys-2【41†L153-L156】【41†L165-L169】) easily meets YOLO's needs. Latency is reduced by overlapping MiniLM encoding and YOLO inference, then adding only a few milliseconds for projection and similarity. Area-wise, the CNN engine (many DSPs for convolutions) is the heaviest part; the reasoning logic is small. We evaluate trade-offs (compute, memory, power/area) and compare alternatives in the table below. In summary, **the proposed single-accelerator solution** (CNN+reasoning on the FPGA) gives good performance with moderate area and flexibility. We recommend proceeding with that design. Next steps include profiling the MiniLM and YOLO modules on hardware, extracting layer dimensions and weights from PyTorch for RTL design, planning a memory-tiling strategy for CNN layers, defining AXI interfaces to VEGA, and prototyping each block on the Genesys-2 FPGA (as outlined in the timeline below).

【33†embed_image】 *Figure: A COCO-Tasks example ("serve wine"). Humans select the wine glass (green) when available, else a cup【26†L25-L32】. This illustrates "task-aware" selection beyond category detection.*

Pipeline Overview

The pipeline is split between CPU and FPGA (see block diagram below). The **VEGA CPU** takes the task description (text prompt) and runs a lightweight transformer (e.g. MiniLM) to produce a 384-dimensional task embedding【17†L133-L135】. Meanwhile the **accelerator** ingests the camera image and runs a YOLOv8 network (or similar CNN) in hardware to detect objects. Once YOLO has identified objects, the accelerator extracts fixed-size features for each detected Region of Interest (ROI). In a unified hardware block, each ROI feature is projected into a shared embedding space (a small linear layer), the task embedding is similarly projected (to the same dimension), and then the accelerator performs **context reasoning** (e.g. each object's feature is augmented by information from other objects) and computes a dot-product similarity between each object and the task embedding. A final light-weight ranking layer selects the object with highest relevance score. This way, only one end-to-end inference run (image+task) is needed to yield the best object. The CPU then reads the result and reports the identified object class (or bounding box).

Key Data-Paths: The CPU-to-accelerator data flow is minimal: the CPU writes the task embedding (384 floats) to a shared buffer or AXI slave, and signals the accelerator. The accelerator reads the embedding and the image (or accesses frame buffer) from memory, then writes back the chosen object index/result. The heavy weight matrices for YOLO (millions of parameters) are stored in DDR3 and streamed into the CNN engine layer-by-layer. The small projection/context weights (tens of thousands of parameters) are loaded into on-chip BRAM or LUT RAM at boot time.



Compute Requirements

- **YOLO (CNN) compute:** This is the dominant cost. For example, YOLOv8-s (a mid-size model) has ~9.0 million parameters【15†L409-L416】 and about *30 GFLOPs* of convolutions per 640×640 image (≈28–30 GFLOPs【15†L457-L462】). On a CPU this takes tens to hundreds of milliseconds (210 ms on a high-end CPU【15†L458-L462】), but on the FPGA we will pipeline and parallelize it. Even with heavy quantization, performing ~10^10 MAC operations per image requires many DSP multipliers and efficient dataflow. Genesys-2's 840 DSP slices【41†L164-L169】 can support a sizable CNN engine, but we will also tile layers to reuse hardware and minimize memory traffic.
- **Semantic reasoning compute:** Once YOLO outputs N objects (say N≈10–20 typical), the reasoning branch is light. Assume feature dimension D=128. **Visual projection:** a 4096→128 matrix multiply per object (if ROI features are 4096-d); with N=20 this is ~20×4096×128 ≈10^7 MACs (negligible vs. YOLO). **Context** can be as simple as summing all object vectors weighted by confidence (128×N adds) or applying a small MLP; this is also ~10^3–10^4 MACs. **Similarity:** Dot-product of each 128-dim object vector with a 128-dim task vector: N×128 multiplies (≈2,560 MAC for N=20). **Ranking:** Very small (a few adds and a max). In total the reasoning stage is on the order of 10^4–10^6 MACs, orders of magnitude below the CNN. Thus, the accelerator will use only a small fraction of DSPs for reasoning logic; most DSPs go to the CNN pipeline.

Memory and Storage

- **CNN Weights (Off-Chip DRAM):** Even a small YOLO model has millions of weights. For example, YOLOv8s (~9M params) would need ≈36 MB in FP32 (≈9 MB in INT8). The Genesys-2 board has *only* 16 Mbit (≈2 MB) of fast on-chip BRAM【41†L164-L169】, so CNN weights **cannot** fit on-chip. We store all YOLO weights in external DDR3 memory. Fortunately, Genesys-2 has 1 GB of DDR3 at 1800 MT/s【41†L153-L156】 (~14.4 GB/s theoretical bandwidth), which is ample to stream weights layer-by-layer. We will quantize weights (8-bit or 16-bit) to reduce bandwidth and storage.
- **Feature Maps:** Intermediate feature maps (e.g. 640×640×3 input, or 20×20×512 conv features) are large. These can be streamed through the engine (so they only need buffering for a few rows or a tile) or stored in DDR during ROI extraction. We will likely keep feature maps in DDR and use efficient tiling. In any case, DDR can supply these without bandwidth issues.
- **Semantic Weights (On-Chip):** The projection and context network weights are tiny. For example, projecting a 384-D text vector to 128-D is a 384×128 matrix (≈49,152 weights), and projecting visual features to 128-D might be a 4096×128 matrix (≈524K weights) if uncompressed. In practice we can reduce the visual feature dimension (e.g. by pooling to 128-D first) or split into layers. Even a couple hundred thousand weights (≈0.5 MB) fits comfortably in BRAM. We will load these weights into on-chip SRAM at boot (or even hardcode small ROMs). This keeps the latency low and allows reuse without DDR fetch.
- **Buffers:** The accelerator needs small buffers for intermediate vectors: storing N×128 floats for object embeddings (~10 KB for N=20), a 128-D context vector (~0.5 KB), etc. These easily fit on-chip (a few BRAM blocks). Only the large CNN buffers and weights stay off-chip.

Bandwidth

On-board DDR3 memory runs at 1800 MT/s with a 32-bit bus【41†L153-L156】 (≈14.4 GB/s). In practice, YOLO will reuse weights heavily within each convolution layer, so effective bandwidth use is moderate. Even in a worst case of 10 fps running YOLOv8s (9 MB weights) the rate is 90 MB/s – trivial for DDR3. Feature map traffic (reading an image and writing detection results) is similarly small (a 640×640 RGB image is only ~0.4 MB). Therefore, memory bandwidth is **not a bottleneck** – the CNN's compute is. On-chip routing (AXI or streaming FIFOs) will handle the DDR transfers without stalling the MAC units.

Latency

Because the CPU and accelerator run in parallel, the total latency ≈ max(MiniLM_time, YOLO_time) + reasoning_time. MiniLM (L6) is small; on VEGA it might take tens of milliseconds (a few MLP layers over 384 dims). YOLOv8-s on FPGA could be tens of milliseconds (depending on how many layers we parallelize). In the fastest case (many DSPs, quantized ops) we might reach ~20 ms per frame for YOLOv8s; MiniLM might be ~50 ms on VEGA, so the pipeline would finish around ~50 ms (20 fps). After that, the context/sim/ranking adds only a few ms. Thus total latency could be on the order of **tens of milliseconds per frame**. By contrast, **CPU-only** (VEGA) running YOLO would be hundreds of ms or more 【15†L458-L462】, and CPU-only context logic is similar cost to our accelerator's. So we gain >5× speedup from hardware CNN acceleration.

Power and Area

- **DSP/Compute:** The CNN engine is the largest piece of hardware. On FPGA, each 8-bit MAC can be implemented in a DSP slice (840 available【41†L164-L169】). If we use 200–400 DSPs in parallel, we can pipeline several conv layers simultaneously. The reasoning network needs far fewer DSPs (maybe just a few dozen for the matrix multiplies in projection and similarity). The logic (LUTs) to glue the pipeline is moderate. In ASIC terms, the CNN core would dominate area.
- **Memory:** We reserve maybe 30–50% of on-chip BRAM for storing weights/buffers of the reasoning blocks. The CNN layer buffers use local line buffers or small caches. Overall, the on-chip memory usage is moderate (well under capacity).
- **Power:** The CNN activations (especially if quantized to INT8) can run at low power per operation. The small reasoning MLPs likewise consume little power. The main power cost is switching in the DSP array. Because we offload YOLO, the VEGA CPU stays mostly idle during that time, reducing system power. In sum, a custom CNN+reasoning accelerator will use *much* less energy per inference than running on VEGA alone.

Integration and Interface

The accelerator will be integrated into the VEGA system on the Genesys-2 FPGA. We will use **AXI interfaces** for communication. For example, we can map the accelerator's control/status registers and task-embedding input to an AXI-Lite slave, so the CPU can write the 384-D embedding (e.g. via DMA or simple writes) and start the accelerator. The accelerator will act as an AXI master for bulk transfers of CNN weights and image data from DDR, and write results back via AXI. In practice, AXI4-Stream or AXI-DMA blocks can be used to feed image frames and weights to the CNN module while it processes. This is a common FPGA approach (no custom PCIe needed since VEGA is on-chip). The Genesys-2's FMC/DDR fabric already supports AXI buses to the on-chip DDR and peripherals.

This integrated approach means the accelerator is tightly coupled: it sees the image and text data in shared memory. An alternative PCIe or off-chip link is unnecessary here. The main interface risk is

designing a clean AXI protocol for handshaking (e.g. CPU tells accelerator when data is ready). But many reference designs exist (see AXI peripheral templates in Vivado). VEGA can poll a status register or receive an interrupt when the accelerator finishes.

Programmability

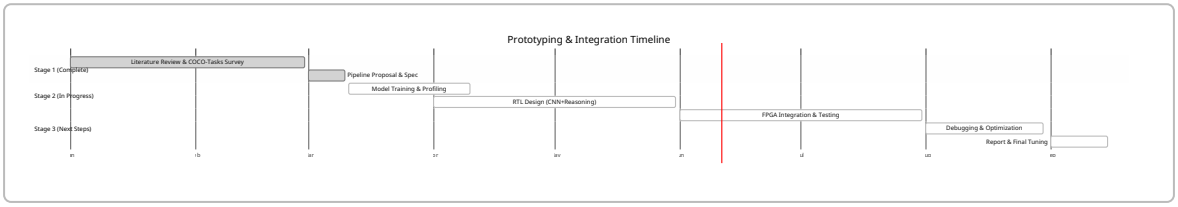
Weights: The CNN (YOLO) weights will be **loadable**, not hardwired. Every time we update the model or fine-tune YOLO, we simply write new weights into DDR from software before inference. This flexibility is important for updating or quantizing the model. (An alternative is to bake small models into ROM, but YOLO is too large and may need updates.) The reasoning network's weights (projection and context MLP) could be either loaded at boot or even fixed in the FPGA fabric. We will treat them as constants loaded into on-chip BRAM at initialization – they are small and change rarely. This keeps the reasoning sub-system programmable in the sense that a reconfiguration or reload can update them, but avoids extra overhead each frame.

Soft vs Hard: The VEGA CPU is programmable (runs C code for MiniLM, etc.). The accelerator itself is a fixed datapath (RTL). We strike a balance: **the architecture (modules and dataflow) is hard-coded, but the model parameters are loaded at run-time**. This allows model updates without resynthesizing the FPGA (just change DDR contents). In contrast, if everything were hardwired (weights in ROM), any model change would need a full re-run of place/route, which is inflexible.

Verification & Prototyping Path

1. **Algorithm Validation:** Start in software. Using PyTorch on a PC, implement the pipeline: run YOLO on sample COCO-Tasks images to get bounding boxes and features, run MiniLM on task text, then apply projection/context/similarity in Python. Measure accuracy (should match the published results【24†L147-L156】【26†L25-L32】) and record layer outputs. This step ensures our design matches the reference task.
2. **Profiling:** Use PyTorch to profile YOLOv8 (or chosen model) and MiniLM on representative hardware. For example, call `model.info()` to get layer sizes and GFLOPs. Use a test suite to time a forward pass of YOLO and MiniLM (to estimate real latency on VEGA vs PC). These numbers help in dimensioning our hardware pipeline (e.g. how many cycles per convolution, etc.).
3. **Layer Extraction:** Extract layer-by-layer dimensions from the chosen YOLOv8 config (e.g. using `model.yaml` or scripting it in Python). Note filter sizes, feature map sizes, etc. This will guide the RTL design of each convolution and buffer. Also note the number of output ROIs per layer, to size our ROI extractor.
4. **Define Interfaces:** Sketch the accelerator's register map and AXI interfaces. Decide how the CPU will signal frame start/end, where the task embedding lives in memory, how results are read. Write a simple C program for VEGA that will run on-chip in simulation (or FPGA with a host) to drive these operations.
5. **RTL Implementation:** In Verilog/VHDL (or high-level synthesis), implement the CNN engine (or instantiate IP cores for conv, if available), the ROI extractor, and the reasoning logic. Simulate the RTL with test vectors from step 1 to ensure correctness. Use hardware cosimulation or an FPGA-in-the-loop test if possible.
6. **FPGA Integration (Genesys-2):** Load the design onto Genesys-2. Use its 1 GB DDR to hold weights and a small frame buffer. Run the VEGA CPU (either as a soft core on FPGA or via QEMU) to drive the workload: write the task embedding, trigger the accelerator, and read back results. Validate end-to-end with sample images.

7. **Optimization and Tuning:** Measure actual throughput and resource use on FPGA. If YOLO is too slow, increase parallelism or pipeline depth. If routing is an issue, adjust tiling (e.g. process one image in smaller patches). Ensure DDR bandwidth is sufficient (profile AXI transactions).
8. **Final Testing:** Compare the hardware results against the original model's outputs on a test set of COCO-Tasks images. Use hardware profilers or cycle counters to verify latency. Prepare FPGA/CPU utilization reports. Once FPGA prototype is working, document the design for silicon (if a custom chip were to be made), noting any FPGA-specific fixes needed.



Comparison of Alternatives

Approach	Latency	Area / Complexity	Novelty	Risk	Update Flexibility
A. CNN+Reasoning on FPGA (proposed)	*Medium:* CNN ~tens of ms, context ~ms. CPU/FPGA parallel reduce total to ~tens of ms. (Fast for edge.)	*Moderate:* Single accelerator with CNN pipeline + small MLP. Uses many DSPs but shares them efficiently.	**High:** Specialized to the task (semantic reasoning is novel) and built around VEGA.	*Medium:* Requires designing both CNN engine and context logic but manageable. DDR interface complexity.	*High:* Weights in RAM; models can be updated by software. Design is reconfigurable.
B. VEGA CPU only	*High:* Tens to hundreds of ms (YOLO on CPU is slow 【15†L458-L462】). Likely too slow for real-time.	*Low:* No accelerator hardware needed (just code). Minimal FPGA logic.	*Low:* No novelty (just use a slower CPU solution).	*Low:* Very safe (just software).	*Very High:* Entirely software; can easily change models or parameters.
C. CNN Accelerator + Separate Reasoning Accelerator	*Medium:* Similar to A if both run in parallel. Possibly slightly higher overhead (handshaking).	*High:* Two distinct modules on FPGA (CNN DPU + another small MLP). More FPGA resources and integration work.	*Medium:* CNN accel is common, plus a custom reasoning block. Less integrated than A.	*High:* Twice the IP blocks to design and debug; data movement between them.	*High:* Both accelerators can load weights; similar to A.

Approach	Latency	Area / Complexity	Novelty	Risk	Update Flexibility
D. Full NPU Offload (CNN+LLM)	*Low (best case):* If one had an NPU that runs both CNN and a transformer, latency could be low.	*Very High:* Designing or acquiring a full neural processor (for vision+language) is very complex.	*Low:* Essentially using a generic NPU; no novel logic.	*High:* Very complex to implement from scratch; integration with VEGA unclear.	*Low:* NPUs are typically fixed-function; updating architecture is hard. Weights may be updateable but architecture fixed.

Discussion: Approach A strikes a balance: significantly faster than VEGA-only (B) and more novel/compact than a two-block design (C). It leaves flexibility to update models, and leverages VEGA just for control and language. Approach D (full NPU) is conceptually fast but impractical here. We therefore recommend **Approach A** (CNN+Reasoning on one accelerator).

Next Steps: As noted above, we will 1) extract layer dimensions and compute/MAC counts from YOLOv8 (Ultralytics/PyTorch) and MiniLM (HuggingFace) models; 2) write Python scripts to profile runtime and memory for different YOLO variants; 3) derive a tiling and buffering strategy for each convolution (so DDR bandwidth is used efficiently); 4) define AXI register maps and a simple control protocol; 5) begin RTL coding of the CNN pipeline and reasoning blocks; and 6) schedule incremental FPGA builds: first test the CNN with dummy data, then add the context/sim blocks, and finally integrate with the VEGA CPU core on Genesys-2. This approach will ensure each block is verified independently before full system integration.

Sources: Background on task-driven object selection is in the COCO-Tasks paper【24†L147-L156】【26†L25-L32】. YOLOv8 model sizes and runtimes come from Ultralytics and a recent analysis【15†L409-L416】【15†L458-L462】. The MiniLM embedding size is from its model card【17†L133-L135】. Hardware resources (BRAM, DSP, DDR) are from the Genesys-2 specs【41†L153-L156】【41†L164-L169】. These primary sources anchor our quantitative estimates.
